



DWH Analyst



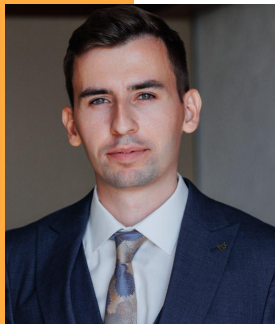
Проверить, идет ли запись

Меня хорошо видно && слышно?



Тема вебинара

Оркестрация: Dagster и DBT



Алексей Железной

Senior Data Engineer/Architect

Магистратура - ФКН ВШЭ

Руководитель курсов **DWH Analyst, ClickHouse для инженеров и архитекторов БД** в OTUS

Преподаватель курсов **Data Engineer, DWH Analyst, PostgreSQL** и пр. в OTUS

[LinkedIn](#)

Правила вебинара



Активно
участвуем



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара



Знакомство с Dagster

Варианты установки

Интеграция с dbt

Рефлексия

Цели вебинара

1. Познакомиться с инструментом оркестрации данных Dagster
2. Изучить принципы развертывания и настройки
3. Обсудить возможности интеграции с DBT

Знакомство с Dagster

Dagster

Dagster — это оркестратор, предназначенный для организации конвейеров обработки данных: ETL, проведения тестов, формирования отчетов, обучения ML-моделей и т.д.

- планирование заданий посредством DAG
- конкурент Dagster'a – Airflow.
- Функционал Dagster'a позволяет нам:
 - Создавать хорошо структурированные пайплайны на основе кода Python.
 - Запускать конвейеры по расписанию и по событию.
 - Отслеживать, мониторить и управлять конвейерами

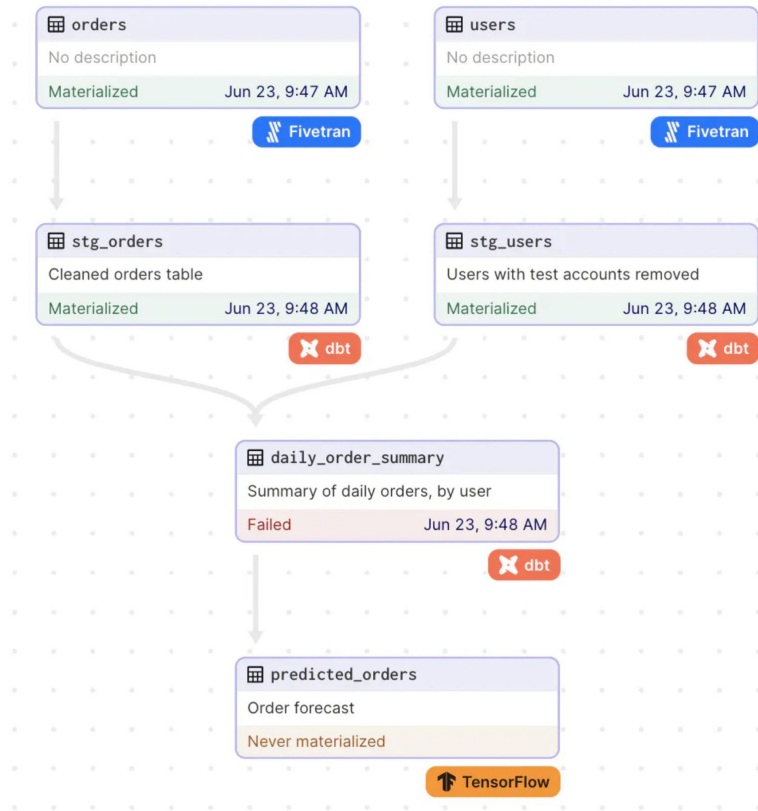


Основные абстракции

- **Assets (Активы)** — это специальная абстракция Dagster'a, которая состоит из двух элементов – функции, которая генерирует контент и физического объекта, который необходимо где-то сохранить (датасет, файл или модель машинного обучения и т.д.).
- **Op (Операции)** и **Job (Задачи)** – **Ops** являются основной единицей вычислений в Dagster'е. Как правило, они выполняют относительно простые задачи, такие как выполнить запрос к БД, что-то посчитать, отправить сообщение и т.д. А **Job** это группировка Операций в единый DAG вычислений.

Почему Dagster?

- **Dagster** моделирует конвейеры в терминах создаваемых и потребляемых ими активов данных, что по умолчанию приносит порядок и наблюдаемость в вашу платформу данных.
- **Активы** в Dagster могут моделировать данные, создаваемые любой системой, например модели dbt, таблицы Snowflake или даже файлы CSV.
- В Dagster вы объявляете **функции на Python**, которые хотите запустить, и **активы данных**, которые эти функции создают или обновляют.



Почему Dagster?

Ориентированный на активы подход Dagster к построению конвейеров данных позволяет:

- Понять, как создается тот или иной актив
- Определить, является ли актив актуальным.
- Стандартизировать лучшие практики. Программно-определяемые активы ([SDA](#), **Software Defined Assets**), концепция Dagster, которая создает активы данных, - это унифицированная абстракция для всех команд, работающих с данными.
 - SDA позволяют упростить сотрудничество и быстро внедрить лучшие практики, такие как языки, специфичные для конкретной области, непрерывная интеграция, локальная разработка и тестирование.

Почему Dagster?

- Упростить отладку. Каждый прогон и вычисление связаны с целью получения данных, поэтому инструменты отладки, такие как журналы, привязаны к производимым активам.
- Полубоваться приятным UI, конечно))

Dagster создан для использования на всех этапах жизненного цикла разработки данных - локальная разработка, модульные тесты, интеграционные тесты, среды хранения, вплоть до производства.

Dagster vs Airflow

Airflow

- С 2015 года, после выпуска Airflow компанией Airbnb, оркестратор быстро завоевал рынок и существенно переиграл уже существующие Luigi(Spotify) и Oozie(Hadoop).
- Огромная поддержка энтузиастов и просто хороших людей)
 - возможность постоянно обновлять и кастомизировать решение под свои нужды
- Понятный интерфейс и принцип работы, устоявшиеся практики по работе с данными
- Продвинутый функционал, такой как branching, triggers, sensors и др.

Dagster vs Airflow

Скачок за 9 лет:

- БОльший уклон в скорость разработки и независимость. Пайплайны данных должны быть написаны, протестированы и развернуты как можно эффективнее и быстрее.
- Создание возможностей самообслуживания и автоматизации, которые позволят большему числу участников создавать активы данных и внедрять их в производство.
- Очень быстрое развитие стека инструментов и правил работы с данными (задачи -> модели dbt, самописные операторы -> предподготовленные коннекторы)
- Возникают вопросы ML, которые тоже многие пытаются внедрить

Но при этом Python все еще во главе таких вопросов)

DAG, да не так. 4 отличия

- Параметры DAG и пользовательский интерфейс
- Тестирование конвейеров
- Управление зависимостями
- Привязка к инфраструктуре

Вопросы?



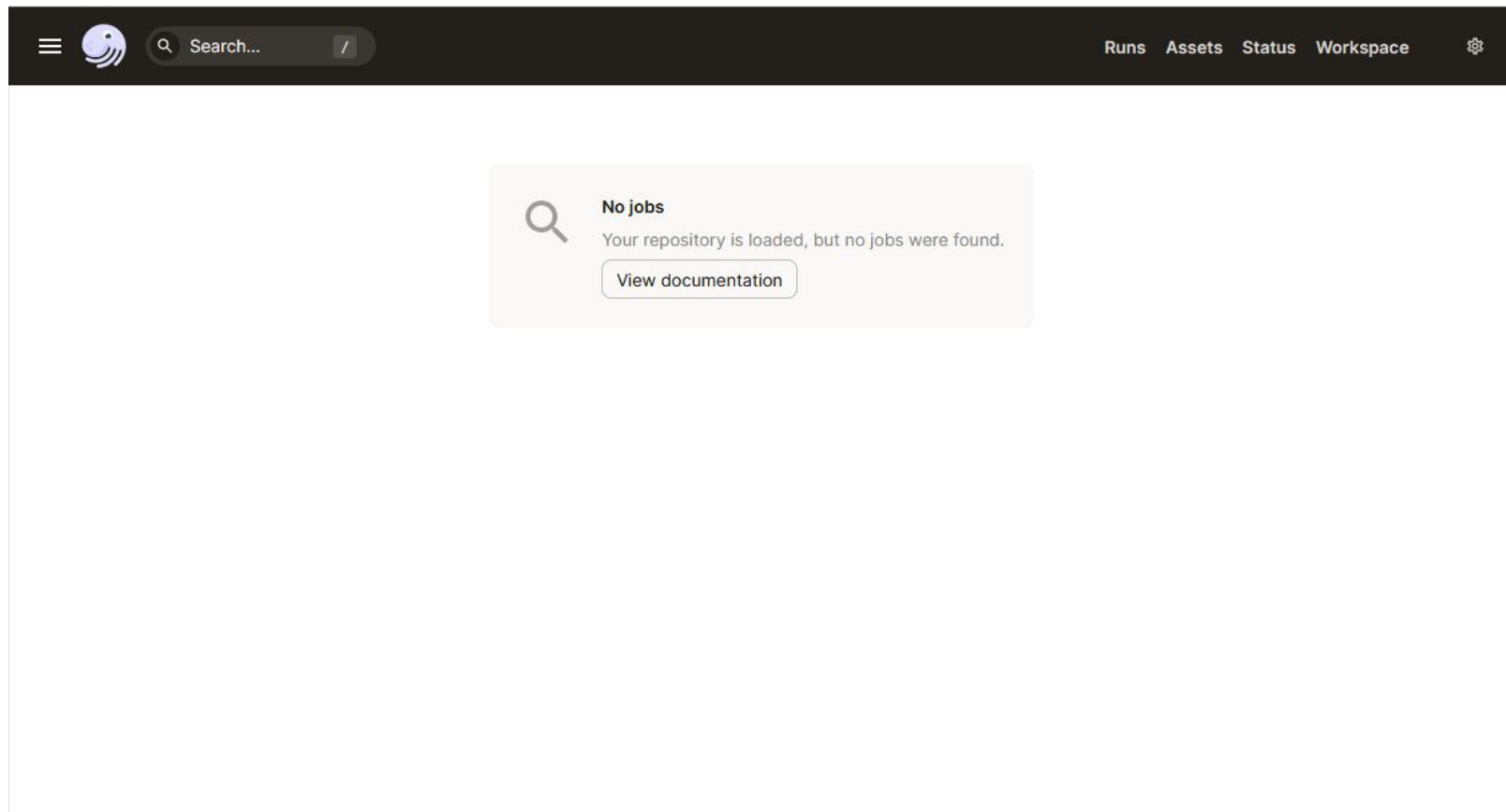
Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Установка и настройка

Base Screen



Assets

☰



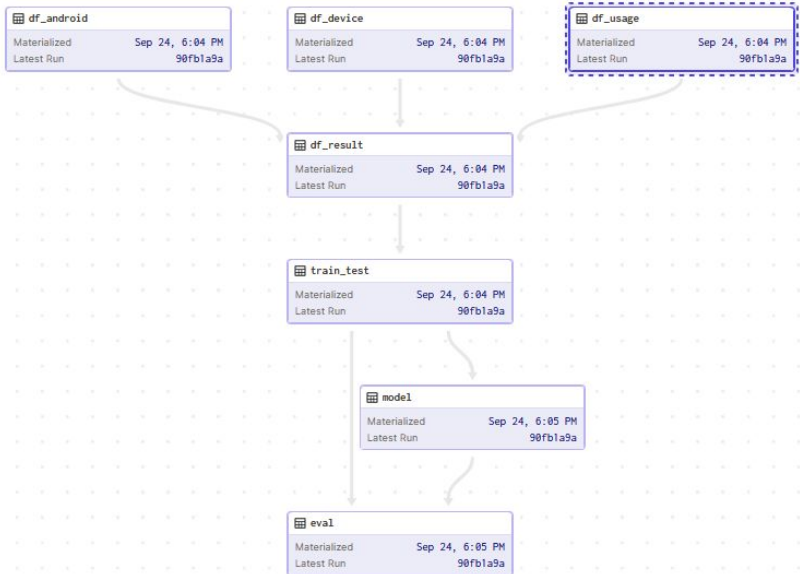
/

Runs Assets Status Workspace ⚙️

Global Asset Lineage

🔄 Reload definitions

0:02 🔄 ✳️ Materialize selected



```
graph TD; df_android[df_android] --> df_result[df_result]; df_device[df_device] --> df_result; df_usage[df_usage] --> df_result; df_result --> train_test[train_test]; train_test --> model[model]; model --> eval[eval];
```

df_usage

[View in Asset Catalog](#)

Materialization in Last Run

Run	Run 90fb1a9a
Timestamp	Sep 24, 6:04 PM
path	E:_test\ml_design_project\tmpv90pi8zf\storage\df_usage

Metadata Plots

No numeric metadata entries available to be graphed.

Config

Any

Type

Any

Assets

@asset

def df_android() -> None:

url = 'https://raw.githubusercontent.com/shanealynn/Pandas-Merge-Tutorial/master/android_devices.csv'

df = pd.read_csv(url)

df = df.rename(columns={'Retail Branding':'brand'})

df.to_pickle('df_android.pkl')

@asset

def df_device():

url = 'https://raw.githubusercontent.com/shanealynn/Pandas-Merge-Tutorial/master/user_device.csv'

df = pd.read_csv(url)

return df

Assets



RunsAssetsStatusWorkspace

Assets

 View global asset lineage

 Reload definitions



0:14 

 Materialize



☐	Asset Key	Defined in	Materialized	Latest run	Actions
☐	 df_android	 ml_design_project@ml_design_project  default	-	-	View details
☐	 df_device	 ml_design_project@ml_design_project  default	-	-	View details
☐	 df_usage	 ml_design_project@ml_design_project  default	-	-	View details

← Previous

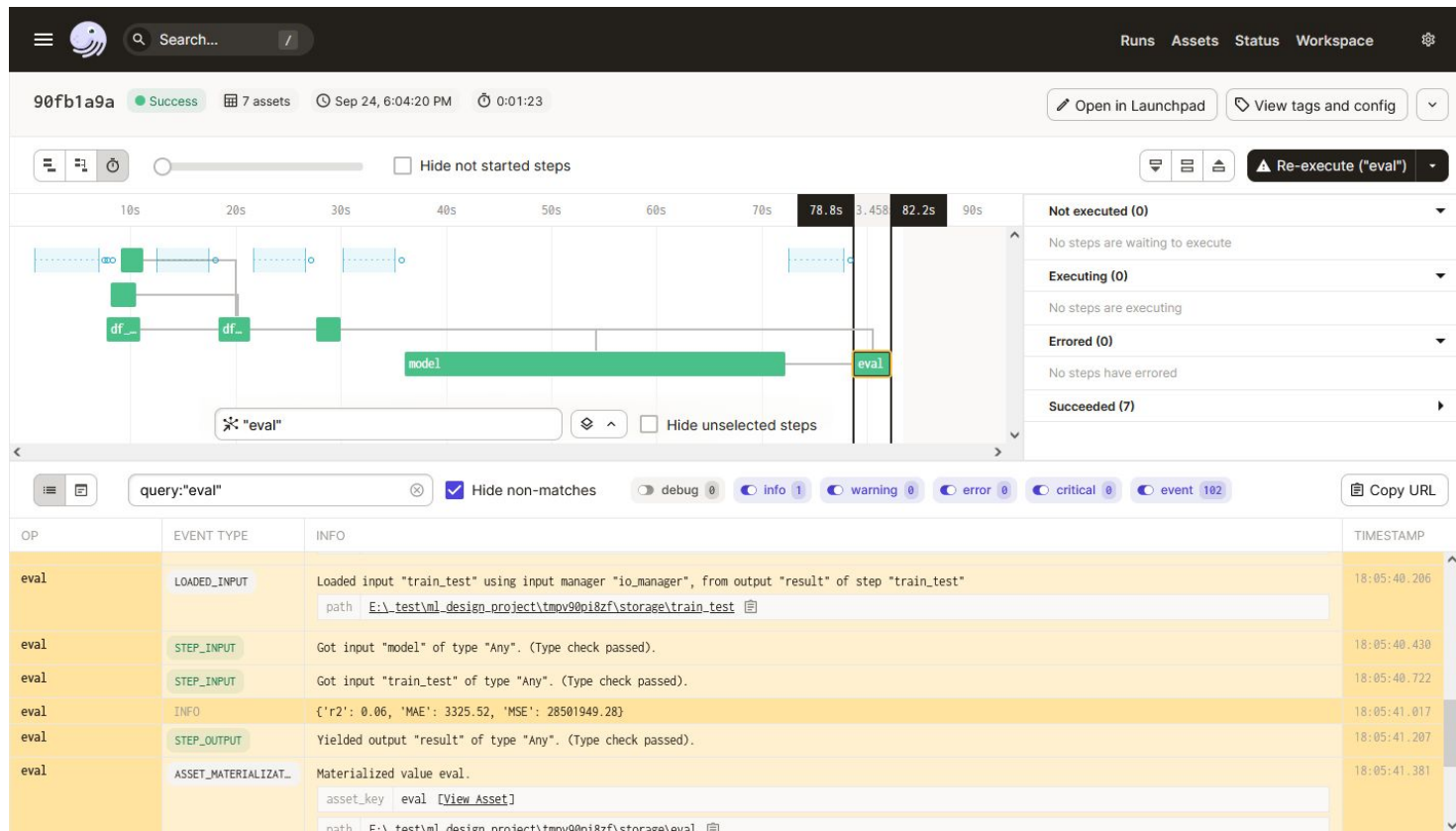
Next →

Логи запуска

The screenshot displays a workflow execution interface. At the top, a search bar and navigation tabs (Runs, Assets, Status, Workspace) are visible. The main header shows the run ID 'e936de6c', a green 'Success' status, the step name 'df_device', and the execution time 'Sep 24, 5:01:32 PM' with a duration of '9.324s'. Below this, there are controls for 'Open in Launchpad', 'View tags and config', and a 'Re-execute (df_device)' button. A timeline view shows the step 'df_device' as a green bar. To the right, a sidebar lists the status of steps: 'Not executed (0)', 'Executing (0)', 'Errored (0)', and 'Succeeded (1)'. Below the timeline, there is a filter bar with a search input and a list of log levels: debug (0), info (0), warning (0), error (0), critical (0), and event (18). The main log table contains the following data:

OP	EVENT TYPE	INFO	TIMESTAMP
df_device	STEP_START	Started execution of step "df_device".	17:01:39.268
df_device	STEP_OUTPUT	Yielded output "result" of type "Any". (Type check passed).	17:01:39.268
df_device	ASSET_MATERIALIZAT...	Materialized value df_device. asset_key df_device [View Asset] path E:_test\ml_design_project\mov90pi8zf\storage\df_device	17:01:39.470
df_device	HANDLED_OUTPUT	Handled output "result" using IO manager "io_manager" path E:_test\ml_design_project\mov90pi8zf\storage\df_device	17:01:40.081
df_device	STEP_SUCCESS	Finished execution of step "df_device" in 1.42s.	17:01:40.240
-	ENGINE_EVENT	Multiprocess executor: parent process exiting after 8.68s (pid: 3536) pid 3536	17:01:41.306
-	RUN_SUCCESS	Finished execution of run for "_ASSET_JOB".	17:01:41.472
-	ENGINE_EVENT	Process for run exited (pid: 3536).	17:01:41.748

Результат



Операции и задачи

The screenshot displays the Argo Workflows web interface. At the top, there's a navigation bar with a menu icon, a search bar, and tabs for 'Runs', 'Assets', 'Status', and 'Workspace'. Below this, the breadcrumb 'serial' and a job identifier 'Job in __repo__...serial@test_op.py' are visible. The main area is divided into 'Overview', 'Launchpad', and 'Runs' tabs, with 'Overview' selected. A search bar for workflow subsets is present. The central workspace shows a workflow diagram with nodes: 'get_random' (output: Any), 'get_multi' (input: rnd: Any, output: Any), 'get_plus' (input: rnd: Any, output: Any), and 'print_result' (input: multi: Any, plus: Any, output: Any). The 'get_multi' node is highlighted with a dashed blue border. On the right, a sidebar provides details for the selected 'get_multi' node, including its invocation, inputs/outputs, execution time graph, status, and definition.

serial Job in __repo__...serial@test_op.py

Overview Launchpad Runs

Type an op subset... (ex: get_random+)

Highlight...

get_random

Any

rnd Any

get_multi

Any

get_plus

rnd Any

Any

multi Any

plus Any

print_result

Any

Info Types

Invocation

get_multi

Inputs

get_random:result → rnd

Outputs

result → print_result:multi

Execution Time

Seconds

1.0

0.5

0

Timestamp

12AM 2AM 4AM 6AM 8AM 10AM 12PM 2PM 4PM 6PM 8PM 10PM 12AM

Execution Status

Last 0 Runs

Definition

Op

get_multi

(rnd: Any) ⇒ (result: Any)

Config

Интерфейс

Search...

Runs Assets Status Workspace

Runs

[All runs](#) Queued 0 In progress 0 Done Scheduled 0:11

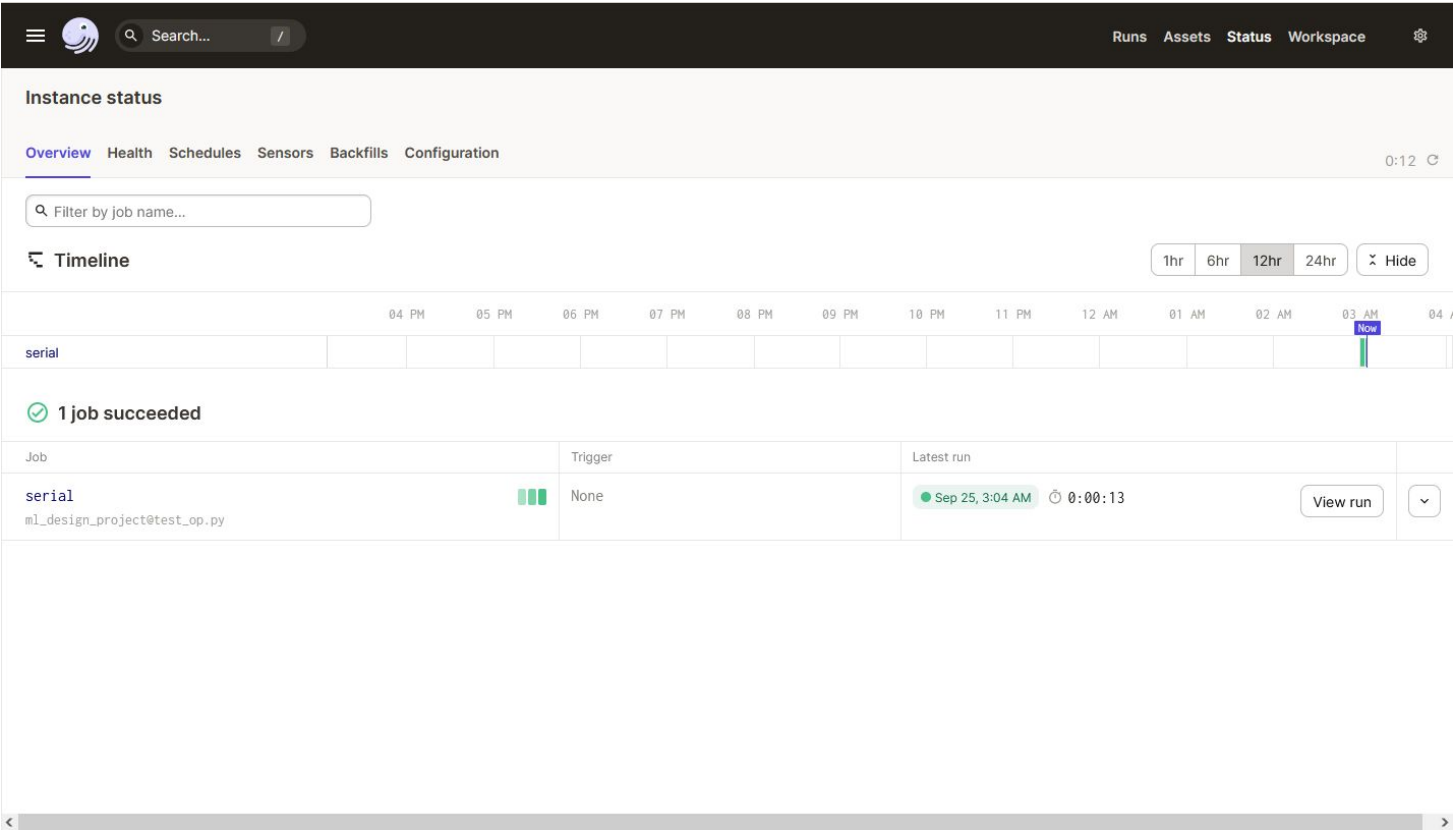
Filter... Actions

☐	Status	Run ID	Job	Snapshot ID	Timing	
☐	● Not started	f9fb2a37	serial parent_run_id: 31ac6c61-65b3-417b-b333-27ff6b9f914d root_run_id: a090102d-74d4-4b8d-8344-84c51120c99f op_selection: * step_selection: "print_result"	f12c43d9	Sep 25, 3:04 AM Starting...	
☐	● Success	e48da2bf	df_result step_selection: df_result	8dc42e03	Sep 25, 3:03 AM 0:00:20	
☐	● Success	31ac6c61	serial parent_run_id: a090102d-74d4-4b8d-8344-84c51120c99f root_run_id: a090102d-74d4-4b8d-8344-84c51120c99f op_selection: * step_selection: get_multi*	f12c43d9	Sep 25, 3:02 AM 0:00:27	
☐	● Success	a090102d	serial op_selection: *	f12c43d9	Sep 25, 3:01 AM 0:00:45	
☐	● Success	a67480cd	+ 7 more step_selection: df_android,df_device,df_result,df_usage,eval,model,tr...	16be2479	Sep 25, 3:01 AM 0:02:38	

← Newer

Older →

Статусы



Filter by job name...

Timeline

1hr

6hr

12hr

24hr

Hide

04 PM

05 PM

06 PM

07 PM

08 PM

09 PM

10 PM

11 PM

12 AM

01 AM

02 AM

03 AM

04

serial

1 job succeeded

Job	Trigger	Latest run	
serial ml_design_project@test_op.py	None	Sep 25, 3:04 AM 0:00:13	View run

Рабочая область



RunsAssetsStatusWorkspace

Workspace

Locations ↻ Reload all

Repository location	Status	Updated	
test_op.py python_file: test_op.py working_directory: E:_test\ml_design_project	Loaded	Sep 25, 2:59:47 AM GMT+3	Reload

Repositories

Repository	Assets	Jobs	Graphs	Ops	Schedules	Sensors
ml_design_project@test_op.py	Assets	Jobs	Graphs	Ops	Schedules	Sensors

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Список материалов для изучения

1. <https://docs.dagster.io/getting-started/quickstart>
2. <https://docs.dagster.io/tutorial>
3. <https://docs.dagster.io/concepts>
4. <https://docs.dagster.io/integrations>
5. <https://habr.com/ru/articles/690342/>

Рефлексия

Рефлексия



С какими впечатлениями уходите с вебинара?

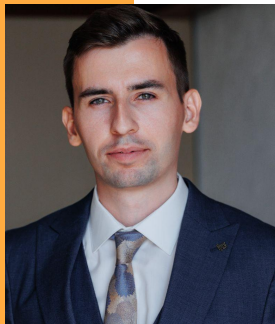


Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!

Приходите на следующие вебинары



Алексей Железной

Senior Data Engineer/Architect

Магистратура - ФКН ВШЭ

Руководитель курсов **DWH Analyst, ClickHouse** для инженеров и архитекторов
БД в OTUS

Преподаватель курсов **Data Engineer, DWH Analyst, PostgreSQL** и пр. в OTUS

[LinkedIn](#)